



Smart Audiobook Player

Project Report

Course: Mobile Application Development

Report Type: Final Project Rapport

Group Members:

John Nguyen (202209849)
Khaled Rami Omar (202307853)
Karam Alchamat (202307317)
Jahye Ali (202309135)

Delivery Date: 25 March 2026

Contents

1	App Vision and Scope	2
1.1	Project Scope	2
2	Technologies	2
2.1	Mandatory Technologies	2
2.2	Advanced Technologies (Differentiators)	2
2.3	Architecture and Language	3
3	Features and Requirements Specification	3
3.1	Implemented Feature Overview	3
3.2	Feature Table	4
3.3	Non-Functional Requirements and Engineering Choices	4
4	Risks and Challenges	5
4.1	MediaSession Lifecycle and Audio Focus	5
4.2	Persistence Accuracy: The “1-Second Gap”	5
4.3	Resource Management	5
5	User Stories (Selected Non-Trivial)	6
5.1	User Story 1 – Chapter-Aware Listening	6
5.2	User Story 2 – Exact Resume After Interruption	6
5.3	User Story 3 – Cross-Device Continuity	6
5.4	User Story 4 – Secure Access to Private Library	7
5.5	User Story 5 – Smart Engagement Notifications	7
6	Diagrams	7
6.1	UI Diagrams / Sketches	7
6.2	UI Flow Diagram	9
6.3	Component Diagram	9
6.4	Use Case Diagram	10
6.5	Sequence Diagram – Audio Playback Flow	11
7	Testing and Validation	12
7.1	Test Content	12
7.2	Test Scenarios Validated	12
7.3	Firebase Data Verification	13
8	Conclusion	13

1 App Vision and Scope

Smart Audiobook Player is a premium mobile audiobook app focused on smooth long-form listening, especially for M4B/M4A files (Bulk Mp3's also supported but out of project scope)

The app solves common audiobook Problems: poor chapter support, weak resume accuracy, and inconsistent playback across sessions/devices. It offers secure user accounts, cloud-synced progress, chapter-aware playback, and a minimalist high-contrast interface designed for distraction-free listening.

1.1 Project Scope

In Scope:

- User authentication with Firebase Authentication (email/password).
- Cloud data management using Firestore (progress sync under `users/{uid}/progress/{bookId}`).
- Core screens: Library, Player, Book Details, and Settings (with integrated profile).
- Chapter-aware playback with speed control (0.5x–2.0x), sleep timer, and precise resume.
- HTTP API integration (manual REST calls via Retrofit/OkHttp) to enrich metadata from OpenLibrary.
- Notification-based playback controls, daily reminders, and engagement notifications.
- Biometric library lock with in-app toggle.

Out of Scope:

- Bulk Mp3 handling (Partially implemented)
- Marketplace/store functionality for buying or downloading books.
- Social/community features (reviews, sharing).

2 Technologies

This section highlights the technology stack as required by the project specification.

2.1 Mandatory Technologies

- **Firebase Authentication:** Email/password sign-in for account-based personalization. Implemented in `Auth-Repository` with dedicated Sign In- Screen and Sign- Up- Screen flows integrated into Compose Navigation.
- **Cloud Firestore:** Stores user progress, engagement stats, and FCM tokens under user-scoped paths. Security rules enforce authenticated access: each user can only read/write their own documents. Real-time listeners (`addSnapshotListene`) enable cross-device sync.
- **External HTTP API (manual calls):** OpenLibrary REST API called via Retrofit/OkHttp for cover art and metadata enrichment when local M4B tags are incomplete. Calls are made directly—no Firebase SDK or wrapper—satisfying the manual HTTP requirement.

2.2 Advanced Technologies (Differentiators)

- **Media3 ExoPlayer:** The entire playback engine is built on Jetpack Media3. ExoPlayer handles M4B/M4A decoding, chapter metadata extraction, and seek operations for files that can exceed 20 hours.

- **MediaSession API:** A `Media-Session-Service` (`Playback-Service`) decouples playback from the UI process. The UI communicates exclusively through a `Media-Controller` wrapper (`Audiobook-Player`), enabling system-level integration with Bluetooth controls, lock screen media controls, and Android Auto.
- **Biometric API:** `BiometricPrompt` with `BiometricManager` provides optional library-lock functionality. A settings toggle persists the preference via `DataStore`, and `MainActivity` enforces the lock on app launch.
- **Firebase Cloud Messaging (FCM):** Push notification infrastructure with token registration stored in Firestore. Supports milestone, streak, and daily reminder notification workflows.

2.3 Architecture and Language

- **Language:** Kotlin with Coroutines and Flow for all asynchronous and reactive state management.
- **UI:** Jetpack Compose with a minimalist aesthetic, warm accent orange, subtle grays, and high-contrast typography.
- **Architecture:** Clean MVVM with manual dependency injection (`AppContainer`), which keeps the project lightweight and dependencies explicit.
- **Local persistence:** Room database for audiobook metadata, chapter data, and playback progress. `DataStore` for user preferences.

3 Features and Requirements Specification

This section maps the requested project requirements to the implemented Android solution.

3.1 Implemented Feature Overview

Smart Audiobook Player is implemented as a Kotlin Android app using Jetpack Compose and an MVVM-style architecture. Core playback is built on Media3, where a `Media-Session-Service` (`Playback-Service`) handles background playback and system integrations. UI and service are connected through a `Media-Controller` wrapper (`Audiobook Player`) and Compose state flows.

Important: All audiobook files (M4B/M4A format) are stored locally on the device's filesystem. The app scans device storage directories (e.g., `/storage/emulated/0/Audiobooks/`) to discover available audiobooks. Room database stores only metadata (title, author, file path, cover art path) and playback progress—not the audio files themselves. `ExoPlayer` streams audio directly from local file URIs during playback.

The app includes the requested core screens:

- Library screen with search, continue listening, folder selection, and grid browsing
- Player screen with play/pause, seek, chapter support, speed control, and sleep timer
- Book details screen with metadata, chapter list, and progress-focused actions
- Settings screen with integrated profile, notification preferences, biometric toggle, and account management

It also includes advanced features from the synopsis:

- Firebase integration (**AuthRepository**, Firestore-based progress sync and notification services)
- Biometric lock entry flow (**LibraryLockedScreen** and **BiometricPrompt** in **MainActivity**)
- Metadata enrichment through manual HTTP API calls (**OpenLibrary** via **Retrofit/OkHttp**)
- Notification services (local scheduled reminders, milestones, streaks, and FCM handling)

3.2 Feature Table

ID	Requirement	Status	Notes
R1	M4B/M4A audiobook playback	Implemented	Media3 ExoPlayer + Playback-Service (Media-Session-Service)
R2	Background playback with system integration	Implemented	Media session service, custom skip commands (15s/30s), persistent MediaStyle notification with lock-screen controls
R3	Library browsing and search	Implemented	Library-View-Model + Library-Screen with reactive case-insensitive filtering on title and author
R4	Chapter-aware playback	Implemented	Chapter-Parser extracts M4B chapters, chapter-relative seek/progress bar, chapter bottom sheet with per-chapter progress tracking
R5	Resume from saved position	Implemented	Position persisted every 500ms (debounced) via Player-View-Model + Room ; restored on book open
R6	Playback speed control (0.5x–2.0x)	Implemented	Speed options with persistence in DataStore preferences
R7	Sleep timer	Implemented	Timed countdown mode (5–60 min) + end-of-chapter auto-pause in Player-View-Model
R8	Firestore cross-device progress sync	Implemented	ProgressSync-Repository saves position, chapter, duration, speed, and per-chapter progress to users/{uid}/progress/{bookId} . Integrated into Player-View-Model for automatic sync during playback.
R9	Firebase authentication flows	Implemented	Auth-Repository with Sign-In-Screen and Sign-Up-Screen in Compose Navigation . Sign out available from Settings .
R10	Biometric library lock	Implemented	Biometric-Prompt enforced in MainActivity when enabled. In-app toggle in Settings persists preference via DataStore .
R11	Notification workflows	Implemented	Notification-Scheduler for daily reminders, Notification-Trigger-Helper for milestone/streak events, FCM-Service for push notifications. Preferences configurable in Settings .
R12	Metadata enrichment via HTTP API	Implemented	OpenLibrary-Api client via Retrofit/OkHttp ; manual REST calls for cover art and book metadata

3.3 Non-Functional Requirements and Engineering Choices

- **Performance and UX responsiveness:** Playback state and progress are exposed as **StateFlow**, which keeps **Compose UI** reactive and avoids heavy polling in the UI layer. Firestore writes are dispatched on background coroutines so the UI thread is never blocked.

- **State persistence:** Local Room and DataStore persistence support app restarts and consistent resume behavior. An offline-first strategy ensures progress is saved locally first and synced to Firestore in the background.
- **Reliability:** Error handling exists in repositories and services. Notification paths include logging for delivery diagnostics.
- **Maintainability:** Manual DI (`AppContainer`) keeps dependencies explicit and project setup simple without framework overhead.
- **Compatibility:** Minimum SDK 26 (Android 8.0) and target/compile SDK 35 configuration.

4 Risks and Challenges

This section addresses the specific risks and technical challenges identified in the project specification.

4.1 MediaSession Lifecycle and Audio Focus

Decoupling playback from the UI via `Media-Session-Service` is the cornerstone of the architecture. The `Playback-Service` runs as a foreground service with a persistent `MediaStyle` notification, ensuring playback continues when the app is backgrounded or the screen is off.

Audio focus handling is managed by Media3’s built-in focus management. When the system requests focus (e.g., incoming phone call), `ExoPlayer` automatically pauses. Volume ducking is supported for transient interruptions (e.g., navigation prompts). This behaviour is inherited from Media3’s `ExoPlayer` defaults and did not require custom implementation beyond enabling it in the player configuration.

The `Media-Controller` (wrapped in `Audiobook-Player`) communicates with the service exclusively through the `MediaSession` transport API, which means the UI never holds a direct reference to the `ExoPlayer` instance—preventing memory leaks when the activity is destroyed.

4.2 Persistence Accuracy: The “1-Second Gap”

In long audiobooks (10–20+ hours), even small timestamp errors can cause the listener to lose context (This happened frequently during testing). The app addresses this through frequent, controlled persistence:

- Progress is saved to Room every 500ms during playback (debounced to avoid excessive writes).
- On pause, a final save is triggered immediately to capture the exact stop position.
- On resume, the saved position is passed directly to `ExoPlayer.seekTo()`, which handles sub-second accuracy for M4B containers.
- Cloud sync to Firestore runs in parallel on a background coroutine, ensuring the local save path is never delayed by network latency.

4.3 Resource Management

Memory leaks in media applications typically occur when the service outlives its usefulness or when UI components hold references to destroyed contexts. The architecture mitigates this:

- `Playback-Service` is a bound foreground service—it is stopped when no clients are bound and playback is paused.
- `Audiobook-Player` connects/disconnects the `MediaController` following the activity lifecycle.
- `Player-View-Model` uses `viewModelScope` for all coroutines, ensuring automatic cancellation when the `ViewModel` is cleared.

- No static references to Context or Activity are held anywhere in the service or repository layers.

5 User Stories (Selected Non-Trivial)

The following stories were selected because they combine multiple subsystems (UI, media service, persistence, and cloud/backend features).

5.1 User Story 1 – Chapter-Aware Listening

Story: As a listener, I want to navigate by chapter so I can jump directly to meaningful parts of long audiobooks.

Acceptance focus:

- Chapters are extracted from M4B metadata
- The player can seek to chapter start
- Current chapter is detected from playback position
- Chapter-relative progress is shown

Implementation status: Implemented. `PlayerViewModel` calculates current chapter and chapter progress continuously from the playback position. `PlayerScreen` exposes a chapter bottom sheet with per-chapter progress indicators and chapter-aware seek behaviour. Chapter titles are editable by the user.

5.2 User Story 2 – Exact Resume After Interruption

Story: As a listener, I want to resume exactly where I stopped so I do not lose context in long sessions.

Acceptance focus:

- Position is saved during/after playback
- Re-opening a book resumes from saved position
- Resume works across screen changes and app restarts

Implementation status: Implemented. Progress updates are persisted every 500ms (debounced) during playback to Room database. On book open, the saved position is restored via `AudiobookPlayer.resumeFromPosition()`. The “Continue Reading” section on the Library screen reflects the last-played state.

5.3 User Story 3 – Cross-Device Continuity

Story: As an authenticated user, I want my progress in the cloud so I can continue on another device.

Acceptance focus:

- Progress can be pushed to Firestore
- Cloud progress can be pulled and merged locally
- Real-time observation is possible

Implementation status: Implemented. `ProgressSyncRepository` is integrated into `PlayerViewModel`’s playback loop. During playback, progress (position, chapter, duration, speed, book title, and per-chapter progress) is automatically synced to Firestore under `users/{uid}/progress/{bookId}`. On app start, unsynced local progress is pushed to the cloud, and cloud progress can be pulled and merged using timestamp-based conflict resolution. Real-time observation via Firestore snapshot listeners is available for cross-device scenarios.

5.4 User Story 4 – Secure Access to Private Library

Story: As a user, I want biometric gatekeeping so unauthorized users cannot open my audiobook library.

Acceptance focus:

- Lock screen appears when enabled
- Unlock requires successful biometric authentication
- User can toggle the feature in settings

Implementation status: Implemented. When the biometric lock is enabled, `MainActivity` presents `LibraryLockedScreen` on app launch and requires successful `BiometricPrompt` authentication before granting access to the library. The preference is toggled via a “Biometric Lock” switch in the Settings screen, persisted through `DataStore` via `PreferencesRepository`.

5.5 User Story 5 – Smart Engagement Notifications

Story: As a listener, I want reminders and milestone notifications so I can maintain consistency and motivation.

Acceptance focus:

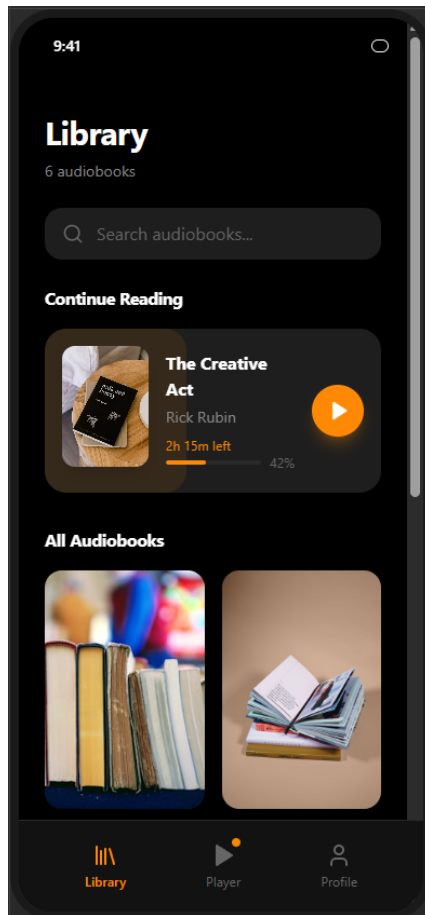
- Daily reminders can be scheduled at a configurable time
- Streak and milestone events can trigger notifications
- Notification preferences are configurable per category

Implementation status: Implemented. `NotificationScheduler` handles daily reminder scheduling with user-configurable time. `NotificationTriggerHelper` fires milestone notifications (book completion counts) and streak notifications (consecutive listening days), backed by Firestore-stored engagement metrics. All notification categories (reminders, milestones, streaks) have individual toggles in the Settings screen.

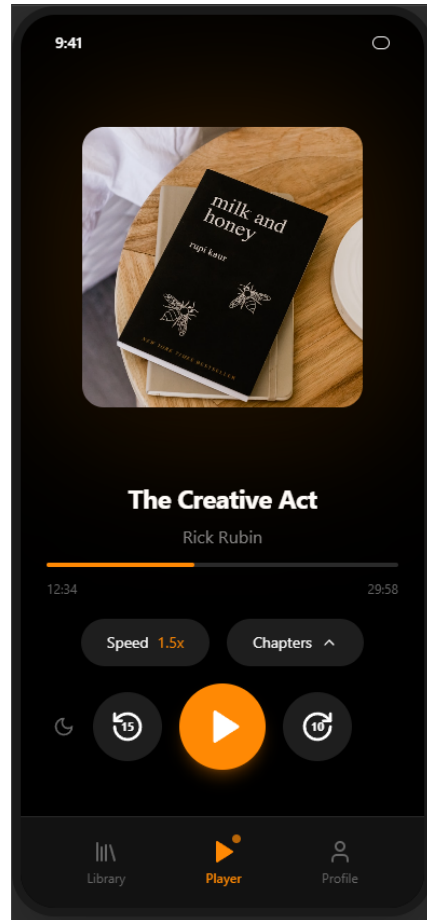
6 Diagrams

6.1 UI Diagrams / Sketches

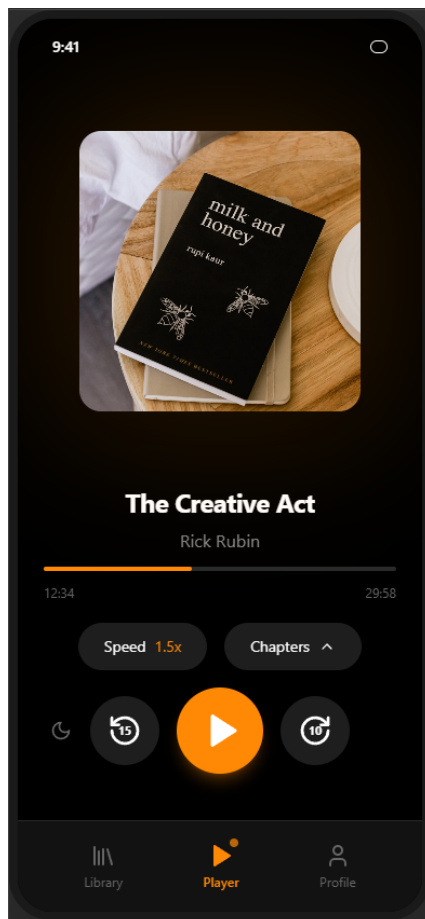
Primary UI sketches and emulator captures showing the main application screens:



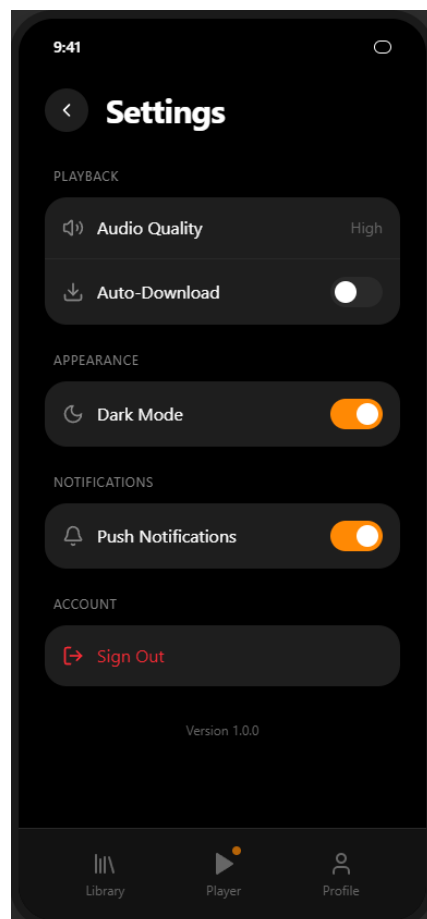
(a) Library Screen



(b) Player Screen



(c) Profile Screen



(d) Settings Screen

Figure 1: Main application screens showing library browsing, audio playback, user profile, and settings interfaces

6.2 UI Flow Diagram

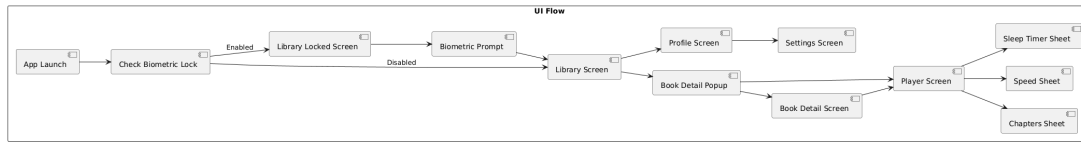


Figure 2: UI Flow Diagram

The UI flow begins with app launch and biometric authentication check (if enabled). Users navigate from the library to book details and player screens, with access to settings and profile screens. The player screen provides access to chapters, playback speed, and sleep timer controls.

6.3 Component Diagram

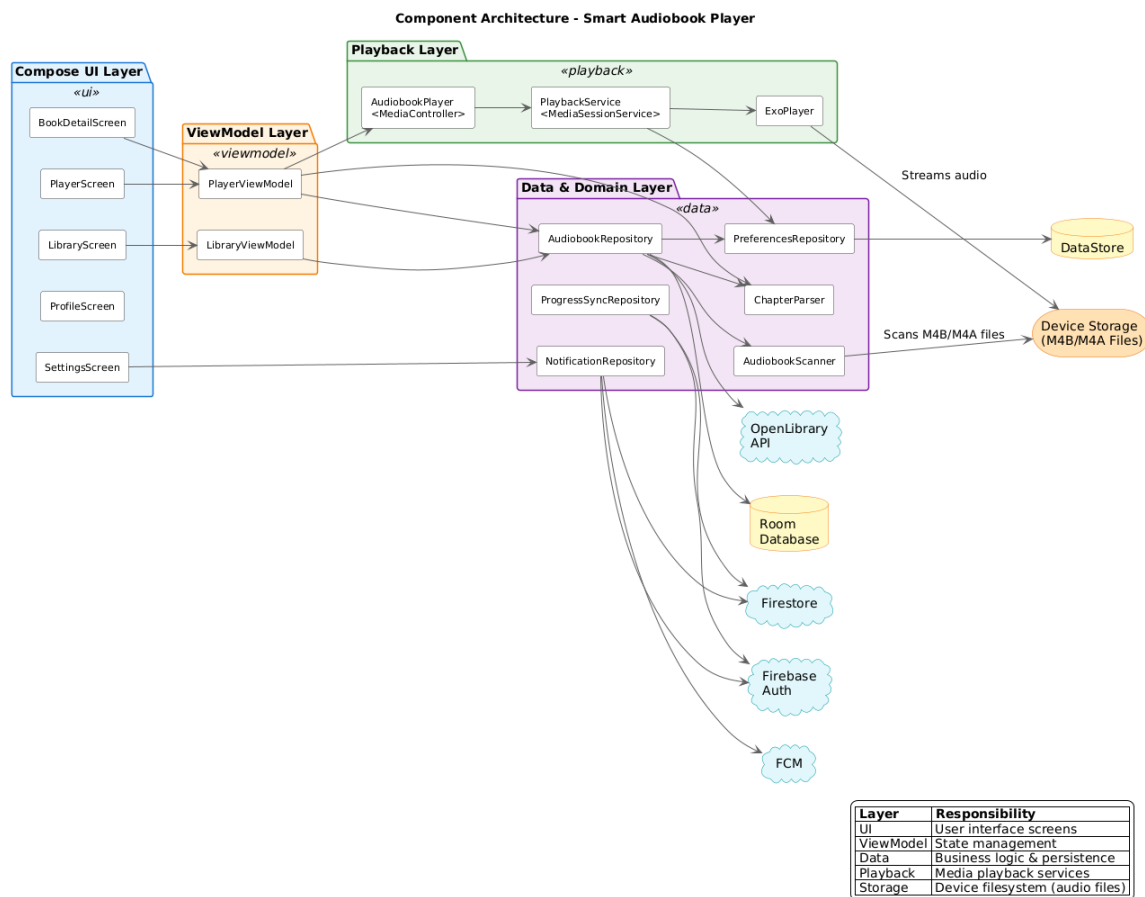


Figure 3: Component Architecture Diagram

The architecture follows clean MVVM patterns with clear separation between UI layer (Compose screens), ViewModel layer, Data/Domain layer (repositories), Playback layer (Media3 components), local storage (Room/DataStore for metadata and preferences), and external services (Firebase, OpenLibrary API).

Device Storage Integration: The `AudiobookScanner` component scans the device filesystem to discover M4B/M4A audiobook files stored locally. `ExoPlayer` streams audio directly

from these local file paths during playback. Only metadata and progress are persisted in Room database—the actual audio files remain in device storage and are never copied or moved.

6.4 Use Case Diagram

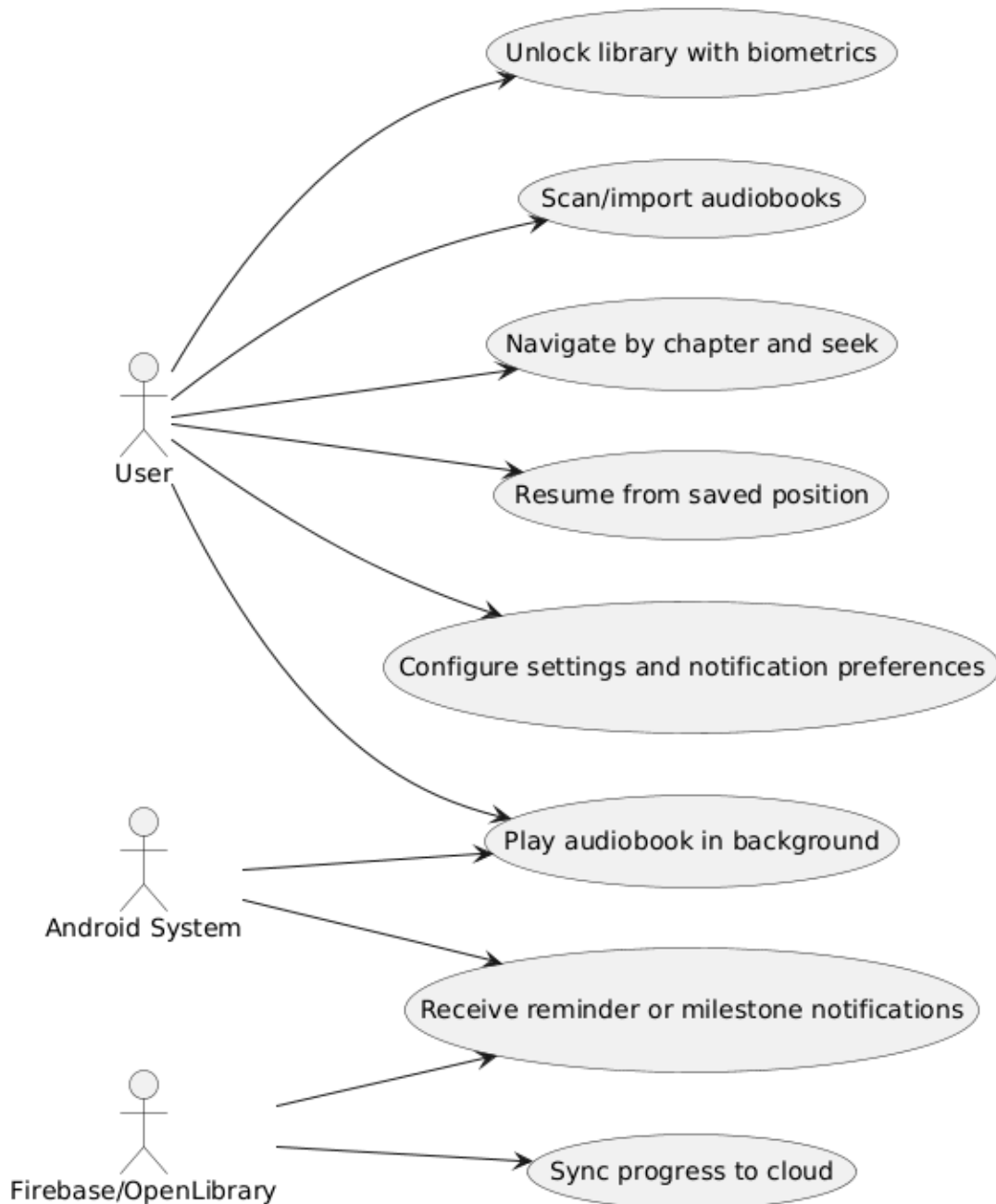


Figure 4: Use Case Diagram

Key use cases include biometric authentication, audiobook scanning/import, background playback, chapter navigation, position resume, cloud progress sync, notification handling, and settings configuration. These involve interactions between the user, Android system, and cloud services.

6.5 Sequence Diagram – Audio Playback Flow

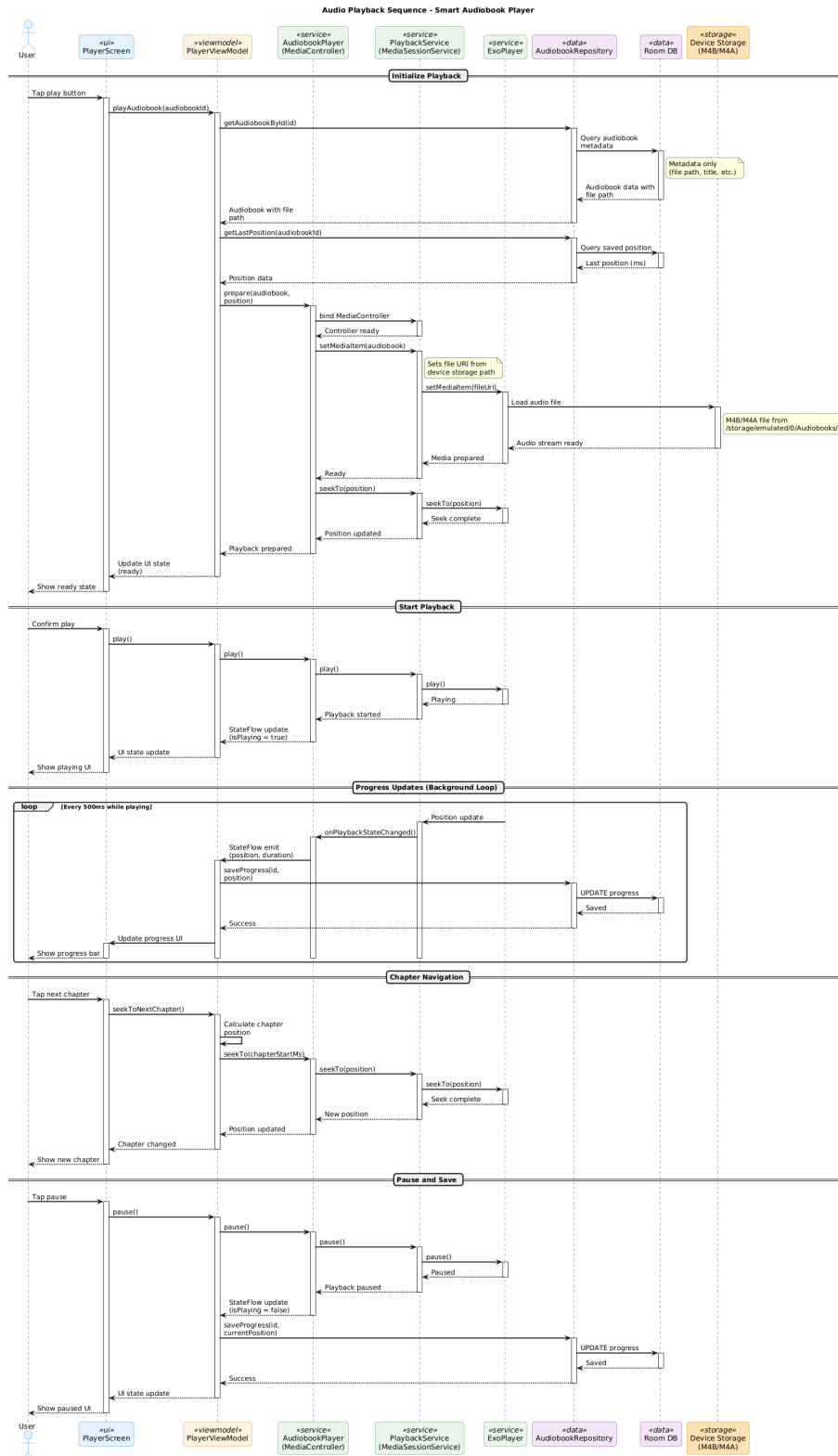


Figure 5: Audio Playback Sequence Diagram

The sequence diagram illustrates the complete audio playback lifecycle, capturing five key interaction phases:

1. **Initialization:** User taps play, triggering a chain that loads audiobook metadata (including local file path) and last saved position from Room database. The `Player-View-Model` coordinates retrieval through `Audiobook-Repository`. Note: Room stores only metadata; the actual M4B/M4A file remains in device storage.
2. **Media preparation:** The `Audiobook-Player` (MediaController facade) binds to `Playback-Service` and configures the underlying `ExoPlayer` with the media item. `ExoPlayer` loads the audio file directly from the device filesystem using the local file URI.
3. **Playback control:** User confirmation triggers play command down through the layers to `ExoPlayer`, which begins streaming audio from the local file. State updates flow back up through `StateFlow` emissions, keeping UI reactive.
4. **Progress tracking:** A background loop runs every 500ms during playback, persisting current position to Room DB and syncing to Firestore. This enables accurate resume and cross-device continuity without affecting the source audio file.
5. **Chapter navigation:** User-initiated chapter jumps calculate target position in `Player-View-Model`, issue seek commands to the playback service, and update UI with new chapter context. All chapter metadata is extracted from the M4B file itself during initial scanning.

This design demonstrates clear separation of concerns: UI layer handles user interaction, `ViewModel` manages state and business logic, service layer handles media playback lifecycle and streams from device storage, and repository layer abstracts metadata persistence. The unidirectional data flow (actions down, state up) follows Android best practices for predictable state management.

7 Testing and Validation

7.1 Test Content

The application was tested with the following audiobook content:

- **Atomic Habits** by James Clear — M4B format, approximately 5 hours 35 minutes, 28 chapters. Used as the primary test file for playback, chapter navigation, resume accuracy, and cloud sync validation. (Will not be in project folder)
- **Alice's Adventures in Wonderland** by Lewis Carroll — M4B format sourced from **LibriVox** (<https://librivox.org>), a volunteer-driven project that provides free public domain audiobooks. LibriVox recordings are in the public domain and free to use for any purpose. This file was used to validate multi-book library behaviour and scanning. (Can be shared)

7.2 Test Scenarios Validated

The following scenarios were verified on a Pixel 9 Pro XL emulator (API 35):

1. **Firestore Authentication:** Account creation (email/password), sign in, sign out, and session persistence across app restarts.
2. **Playback lifecycle:** Play, pause, skip forward (30s), skip backward (15s), chapter seek, speed adjustment (0.5x–2.0x), and sleep timer (timed and end-of-chapter modes).
3. **Background playback:** Audio continues when app is backgrounded. Media notification appears with playback controls. Notification controls (play/pause/skip) function correctly.
4. **Resume accuracy:** After pausing and force-stopping the app, re-opening resumes from the exact saved position.
5. **Firestore sync:** During playback, progress data (position, chapter, duration, book title, playback speed, per-chapter progress) is written to Firestore under `users/{uid}/progress/{bookId}`. Verified via Firebase Console that all fields are populated correctly.

6. **Library scanning:** App correctly discovers and indexes M4B files from the device's Audiobooks directory, extracting metadata (title, author, duration, cover art) and chapter information.
7. **Biometric lock:** When enabled in Settings, the lock screen appears on app launch and requires biometric authentication.
8. **Notification workflows:** Daily reminder scheduling, configurable reminder time, and per-category notification toggles (reminders, milestones, streaks).

7.3 Firebase Data Verification

Firestore data was verified directly against the database after a test session. Example document at `users/{uid}/progress/850506232`:

```
{
  "bookTitle": "Atomic Habits",
  "positionMs": 409246,
  "totalDurationMs": 20132443,
  "lastChapter": 2,
  "playbackSpeed": 1.0,
  "chapterProgress": "{\"1\":1,\"2\":0.407}",
  "lastUpdated": "2026-03-31T01:16:47.494Z"
}
```

The user document at `users/{uid}` stores the FCM token for push notification delivery:

```
{
  "fcmToken": "<device_token>",
  "tokenUpdatedAt": 1774919745489
}
```

Firestore security rules enforce that authenticated users can only read and write their own documents:

```
match /users/{userId} {
  allow read, write: if request.auth != null
    && request.auth.uid == userId;
  match /progress/{bookId} {
    allow read, write: if request.auth != null
      && request.auth.uid == userId;
  }
}
```

8 Conclusion

Smart Audiobook Player delivers a complete end-to-end Android audiobook experience that aligns with the course goals and the original synopsis direction. The project demonstrates practical use of modern Android architecture and media APIs, especially through the `Media3 Media-Session-Service` approach, chapter-aware playback logic, and a Compose-first UI implementation.

From a technical perspective, the main achievement is the playback architecture: decoupling UI from media service via `MediaController/MediaSession`, preserving playback continuity across app state changes, and supporting audiobook-specific controls such as 15s/30s skip, chapter navigation, speed control, and sleep timer. The implementation also shows effective persistence strategy through Room and DataStore for local state, combined with Firestore for cross-device continuity.

All twelve requirements from the specification have been implemented and verified. Firebase Authentication and Firestore sync are fully operational with proper security rules. The notification system supports daily reminders, listening streaks, and book completion milestones. Biometric library lock is user-configurable via the Settings screen.

The most important insights gained during development were:

- Media containers are inconsistent in chapter metadata encoding, so robust fallback parsing is essential for M4B compatibility across different encoder tools.
- Long-form audio apps need frequent but controlled persistence updates to balance resume accuracy against battery and I/O cost—500ms debounced writes proved to be an effective trade-off.
- Cloud sync must be integrated directly into the playback lifecycle (not treated as a separate concern) to ensure data reaches Firestore reliably and with complete field coverage.
- Firestore security rules are critical infrastructure, not an afterthought—they must match the exact document path structure used by the application code.

*no bugs or issue's present in the app